

Bericht Probabilistische Energiedatenvorhersage

Entwicklung eines probabilistischen LSTM/RNN-Modells zur Vorhersage von Photovoltaik-Einspeisung auf Basis von SMARD-Marktdaten

Projektbericht

Valentin Bäumer und Jan-Christian Raabe

2026

Inhaltsverzeichnis

1	Einleitung und Zielsetzung	1
1.1	Welches Problem soll das Projekt lösen?	1
1.2	Interesse und Motivation	2
1.3	Was soll demonstriert werden?	2
1.4	Stand der Technik	2
1.5	Das finale Produkt	3
1.5.1	Qualitative Produktbeschreibung	3
1.5.2	Technische Implementierung und Software-Architektur	6
2	Arbeitspakete	8
2.1	AP1: Recherche nach geeigneten Datenquellen und explorative Datenanalyse	8
2.2	AP2: Modellarchitektur und probabilistisches Training (LSTM vs. RNN)	8
2.3	AP3: Evaluierung, Modellvergleich und Erklärbarkeit	8
2.4	AP4: Containerisierung und Projektdokumentation	9
3	Vorgehensweise und Projektumsetzung	10
3.1	Kick-Off	10
3.2	Zeitplan	10
3.3	Organisation im Projektteam	12
3.4	Umsetzung der Projektarbeit	12
4	Ergebnisse und Diskussion	17
4.1	Explorative Datenanalyse	17
4.1.1	Datensatzstruktur und Vollständigkeit	17
4.1.2	Deskriptive Analyse der Kernkomponenten	18
4.1.3	Mathematische Stationaritätsprüfung	20
4.1.4	Tensor-Transformation für das überwachte Lernen	20
4.2	Qualitativer und quantitativer Vergleich der Architekturen	21
5	Fazit und Reflexion	26
6	Ausblick	28
7	Literatur	29

Abkürzungsverzeichnis

ADF	<i>Augmented Dickey-Fuller</i>
AP	Arbeitspaket
ARIMA	<i>Autoregressive Integrated Moving Average</i>
CSV	<i>Comma-Separated Values</i>
EDA	<i>Exploratory Data Analysis</i>
GARCH	<i>Generalized Autoregressive Conditional Heteroskedasticity</i>
JSON	<i>JavaScript Object Notation</i>
KI	Künstliche Intelligenz
KW	Kalenderwoche
LLM	<i>Large Language Model</i>
LSTM	<i>Long Short-Term Memory</i>
PICP	<i>Prediction Interval Coverage Probability</i>
PNG	<i>Portable Network Graphics</i>
PTH	<i>PyTorch</i>
PV	Photovoltaik
QRNN	Quantils-Regressions-Neuronale-Netze
RMSE	<i>Root Mean Squared Error</i>
RNN	<i>Recurrent Neural Network</i>
SMARD	Strommarktdaten
TFT	<i>Temporal Fusion Transformer</i>

Abbildungsverzeichnis

Abbildung 1.1: Frontend des finalen Produktes	4
Abbildung 1.2: Fortschrittsanzeige	4
Abbildung 1.3: Ergebnis-Reiter	5
Abbildung 1.4: Ergebnis-Dashboard.....	6
Abbildung 4.1: Ergebnisse der EDA – Energieerzeugung.....	17
Abbildung 4.2: Durchschnittliches Tagesprofil	19
Abbildung 4.3: Korrelationsmatrix	19
Abbildung 4.4: Lernkurven beider Architekturen (LSTM und RNN) im direkten Vergleich	21
Abbildung 4.5: PV-Vorhersage (1).....	22
Abbildung 4.6: PV-Vorhersage (2):.....	22
Abbildung 4.7: PV-Vorhersage (3).....	23

Tabellenverzeichnis

Tabelle 4.1: Güteparametervergleich.....	24
---	----

1 Einleitung und Zielsetzung

Aufgrund wetterbedingter Fluktuationen stehen Netzbetreiber durch die fortschreitende Integration volatiler, erneuerbarer Energien vor wachsenden Planungsunsicherheiten. Klassische statistische Modelle liefern meist nur starre Punktprognosen, sodass unerwartete Abweichungen zu einem erhöhten Bedarf an kostenintensiver Ausgleichsenergie führen können, um die Netzstabilität sicherzustellen.

Daher besteht das Ziel dieser Arbeit in der Entwicklung, Implementierung und Evaluierung einer probabilistischen, multivariaten Zeitreihenvorhersage für Photovoltaik (PV)-Erzeugung, die statt einer reinen Punktprognose einen dynamischen Sicherheitskorridor (Konfidenzintervall 80 %) mittels einer Quantil-Regression (Pinball Loss) berechnet.

Dabei fokussiert sich der Gang der Untersuchung auf folgende Aufgaben:

- Vergleich der Architekturen Long Short-Term Memory (LSTM) und Recurrent Neural Networks (RNN): Dabei wird empirisch analysiert, ob ein LSTM oder ein klassisches RNN eine höhere Präzision bei der Modellierung des Unsicherheitsbandes aufweist und sich somit besser für die kurzfristige solare Prädiktion eignet.
- Erklärbarkeit und Transparenz: Mittels der Permutation Feature Importance ist mathematisch offenzulegen, welche physikalischen Wechselwirkungen und Einflussfaktoren die Vorhersage der Modelle maßgeblich steuern, um die Black-Box-Struktur der neuronalen Netze für den Handelseinsatz transparenter zu gestalten.
- Validierung der Dynamik: Im Vergleich mit realen historischen Daten soll demonstriert werden, dass sich das prognostizierte Unsicherheitsband bei volatilen Wetterlagen ausdehnt und sich bei stabilen Bedingungen verengt, um dem Netzmanagement ein verlässliches Instrument zur Risikominimierung anzubieten.

1.1 Welches Problem soll das Projekt lösen?

Das Projekt soll das Problem der Planungsunsicherheit bei volatilen, erneuerbaren Energien (speziell Photovoltaik) lösen. Die Kernproblematik besteht darin, dass herkömmliche statistische Modelle oft nur eine starre Punktprognose liefern. Liegt das Modell beispielsweise wetterbedingt (z. B. durch plötzliche Bewölkung) falsch, müssen Netzbetreiber in Sekunden-schnelle extrem teure Ausgleichsenergie einkaufen, um einen Blackout zu verhindern. Durch

den Einsatz eines probabilistischen multivariaten LSTM und eines RNN berechnet das Modell in dieser Arbeit über eine Quantils-Regression (Pinball Loss) einen dynamischen Sicherheitskorridor (80%-Konfidenz-intervall). Der Netzbetreiber soll daher sofort sehen können, wie sicher sich die künstliche Intelligenz (KI) bei der Prognose ist, um das finanzielle Risiko zu minimieren.

1.2 Interesse und Motivation

Es soll ein multivariates Modell verwendet werden, das die physikalischen Wechselwirkungen im Stromnetz lernt. Mittels der Permutation Feature Importance soll mathematisch messbar und für den Menschen transparent gemacht werden, welche Faktoren (z. B. die Tageszeit) gerade den größten Einfluss auf die Solarprognose haben. Das schafft das notwendige Vertrauen für den echten Handelseinsatz.

1.3 Was soll demonstriert werden?

Es soll gezeigt werden, wie die KI im Vergleich zur Realität abschneidet. Das Modell soll demonstrieren, wie sich das Unsicherheitsband bei wechselhaftem Wetter ausdehnt und bei stabilen Wetterlagen verengt, mit dem Ziel die Planungssicherheit zu erhöhen.

1.4 Stand der Technik

Die probabilistische Zeitreihenprognose fluktuierender erneuerbarer Energien gewinnt zur Vermeidung der kostenintensiven Ausgleichenergien und zur Sicherung der Netzstabilität massiv an Bedeutung. Klassische, statistische Ansätze wie Autoregressive integrierte Moving-Average-Modelle (ARIMA) liefern primär starre Punktprognosen. Zur Unsicherheitsmodellierung etablierten sich sogenannte Generalized Autoregressive Conditional Heteroskedasticity- (GARCH-) Modelle, die die zeitlich veränderliche Volatilität der Residuen abbildet. Solche linearen Verfahren stoßen jedoch bei der Erfassung nicht-linearer meteorologischer Wechselwirkungen auf Grenzen.^[1]

Mit der Entwicklung der Deep Learning Architekturen wurden sogenannte Quantils-Regressions-Neuronale-Netze (QRNN) eingeführt, die statistische Quantilsschätzungen mittels Pinball Loss in neuronale Strukturen integrieren. Das in dieser Arbeit entwickelte Framework erweitert

diesen Ansatz deterministisch um eine zeitliche Sequenzkomponente rekurrenter Netzarchitekturen (LSTM und RNN), um die physikalische Trägheit der Wetterbedingungen abzubilden.^[1]

Im Vergleich zu bereits bestehenden industriellen und akademischen State-of-the-Art-Modellen lässt sich der gewählte Ansatz abgrenzen:

- Das LSTM-basierte DeepAR (Amazon) Framework nutzt eine parametrische probabilistische Vorhersage, indem die Parameter einer hypothetischen Verteilung (Gauß- oder t-Verteilung) geschätzt werden. Die Verteilung der PV-Einspeisung ist durch die harten Nullgrenzen in den Nachtstunden extrem schief und nicht-normalverteilt. Daher bietet der in dieser Arbeit genutzte nicht-parametrische Pinball-Loss-Ansatz mathematisch eine deutlich höhere Flexibilität.^[2,3]
- Sogenannte Temporal Fusion Transformer (TFT) nutzen Aufmerksamkeitsmechanismen zur multivariaten Vorhersage und bieten eine inhärente Erklärbarkeit. Das hier entwickelte Modell erreicht ein ähnliches Maß an Transparenz und Interpretierbarkeit, indem die Black-Box-Struktur der Netze über eine modellunabhängige Permutation Feature Importance mathematisch offengelegt wird.^[4]

1.5 Das finale Produkt

1.5.1 Qualitative Produktbeschreibung

Das im Rahmen dieser Arbeit entwickelte Produkt verarbeitet SMARD Energiemarktdaten im CSV Dateiformat über ein integriertes LSTM und RNN zu einer Prognose. Anstatt einer starren Punktprognose endet die Ausgabe in einer linearen Schicht mit drei Ausgabewerten $q_{0,1}$, $q_{0,5}$ und $q_{0,9}$, die zusammen das Prognoseintervall aufspannen. Somit wird die Unsicherheit in der Prognose explizit sichtbar gemacht.

Das Frontend beinhaltet das Dashboard, das den Nutzer zum Hochladen einer CSV-Datei unter „SMARD CSV-Datei“ auswählen auffordert, siehe Abbildung 1.1. Dabei kann die CSV-Datei entweder durch Drag and Drop oder durch Auswahl von „Browse files“ ausgewählt werden. Die Datenpipeline (mit der Datenverarbeitung, dem Training des LSTM und RNNs) wird durch Auswahl von „Pipeline starten“ gestartet. Diese Auswahl steht jedoch erst zur Verfügung, nachdem durch den Nutzer eine CSV-Datei ausgewählt wurde. Optional kann auch eine CSV-Datei mit Wetterdaten unter Optionale Wetter-CSV ausgewählt werden.



Abbildung 1.1: Frontend des finalen Produktes: Dashboard mit der Aufforderung eine CSV-Datei hochzuladen.

Alle hochgeladenen Dateien befinden sich im Input-Verzeichnis `\smard_daten\input`. Nachdem die Pipeline gestartet wurde, wird der Nutzer regelmäßig durch Updates in der Verarbeitung informiert, siehe Abbildung 1.2:

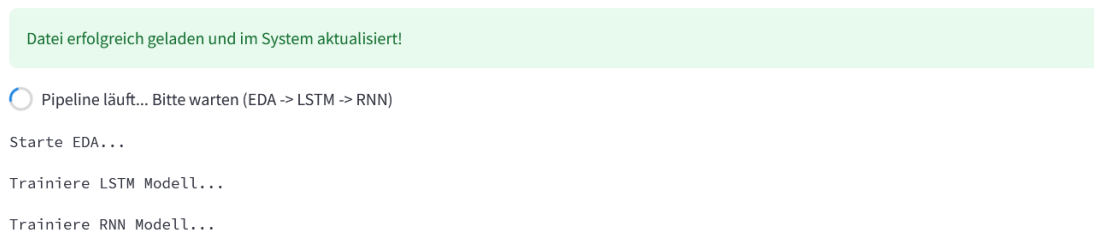


Abbildung 1.2: Fortschrittsanzeige: Statusaktualisierung des Fortschritts.

Nachdem die Pipeline beendet wurde, wechselt das Frontend automatisch in die Rubrik „Überblick“. Dort kann der Nutzer die wichtigsten Metriken einsehen, siehe Abbildung 1.3. Wechselt der Nutzer in die Rubrik „Prognosen und EDA“, werden die Prognosen des LSTM und RNN als Grafiken gezeigt. Darunter befinden sich die Lernkurven und die Ergebnisse der Datenverarbeitung, siehe Abbildung 1.4.

Die Rubrik „Diagnose“ informiert den Nutzer über die Ergebnisse der LSTM/RNN Quantil-Kalibrierungen und bildet tabellarisch weitere Metriken beider Architekturen ab. In der finalen Rubrik „Dateien und Protokoll“ befindet sich eine Auflistung aller erzeugten Dateien, die im Output-Verzeichnis `\smard_daten\output` abgelegt werden. Das Pipeline-Protokoll enthält alle wesentlichen Informationen während des Trainings und spiegelt die Konsolenausgaben der einzelnen Python-Skripte wider. An dieser Stelle besteht die Möglichkeit die LSTM/RNN

Metriken als CSV-Dateien herunterzuladen oder die Pipeline mit einer neuen CSV-Datei wieder zu starten. Bei Neuauswahl einer CSV-Datei werden bereits dort existierende Dateien sowohl im Input- als auch im Output-Verzeichnis überschrieben.

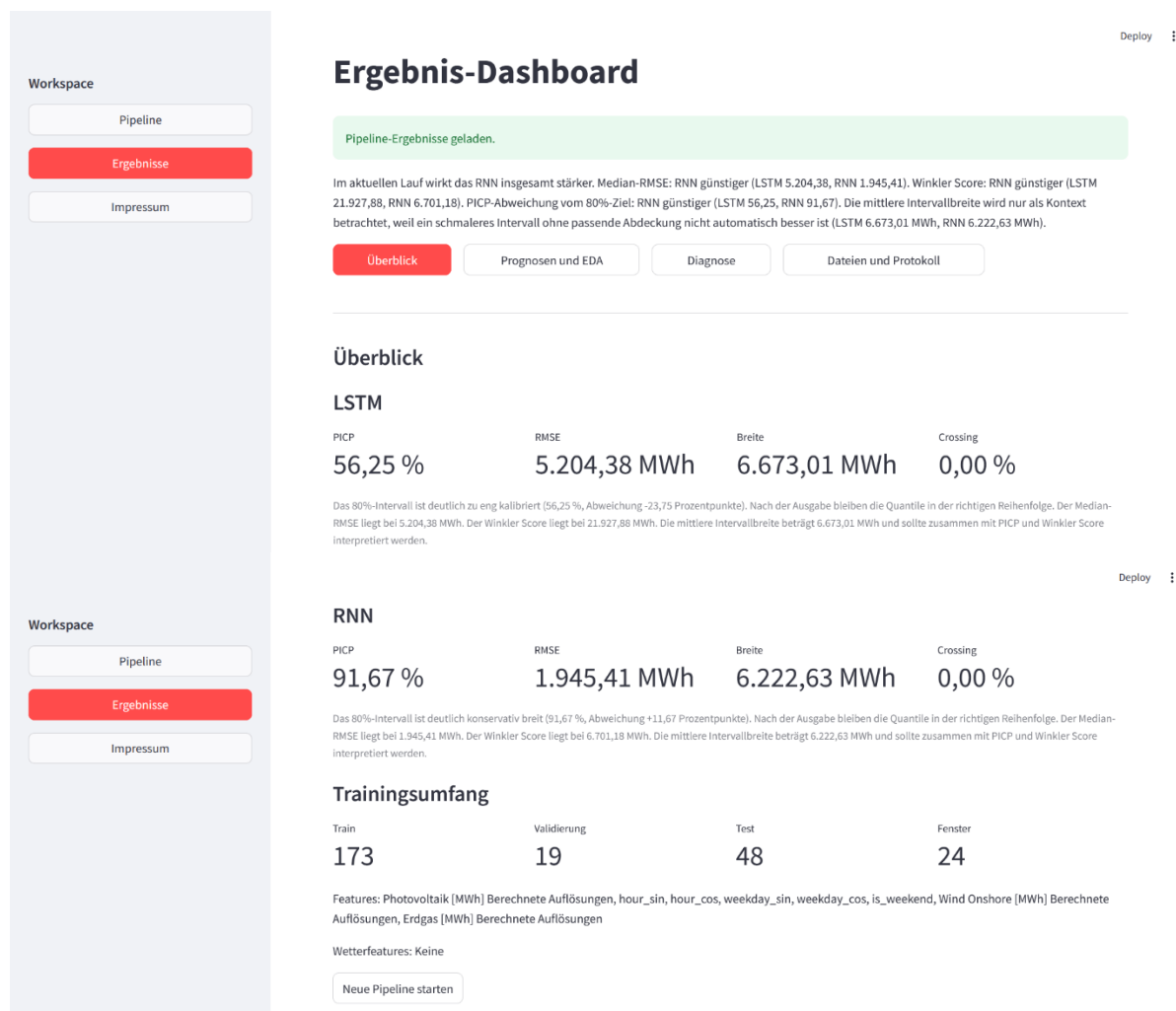


Abbildung 1.3: Ergebnis-Reiter: Ergebnis-Dashboard in der Rubrik „Überblick“ mit allen zugehörigen Metriken.

Die genaue Protokollierung aller zugehörigen Komponenten, dient der Reproduktionsfähigkeit und Dokumentation des Laufes.

Auf der Hauptseite von [SMARD](#) kann direkt die Auflösung der Daten bestimmt werden (z. B. stündliche oder viertelstündliche Auflösung) oder der Zeitraum angepasst werden. Es muss beachtet werden, dass die Rechenzeit der hier vorgestellten Datenpipeline steigt je höher die Auflösung und je länger der Zeitraum gewählt wird, da so mehr Datenpunkte vorhanden sind. Für Demonstrationszwecke empfiehlt es sich daher, einen Datensatz mit einer möglichst groben Auflösung aus einem kurzen Zeitraum auszuwählen. In diesen Szenarien werden auch die

Unterschiede beider Architekturen besser veranschaulicht, siehe Abbildung 1.4 und Rubrik „Prognosen und EDA“.

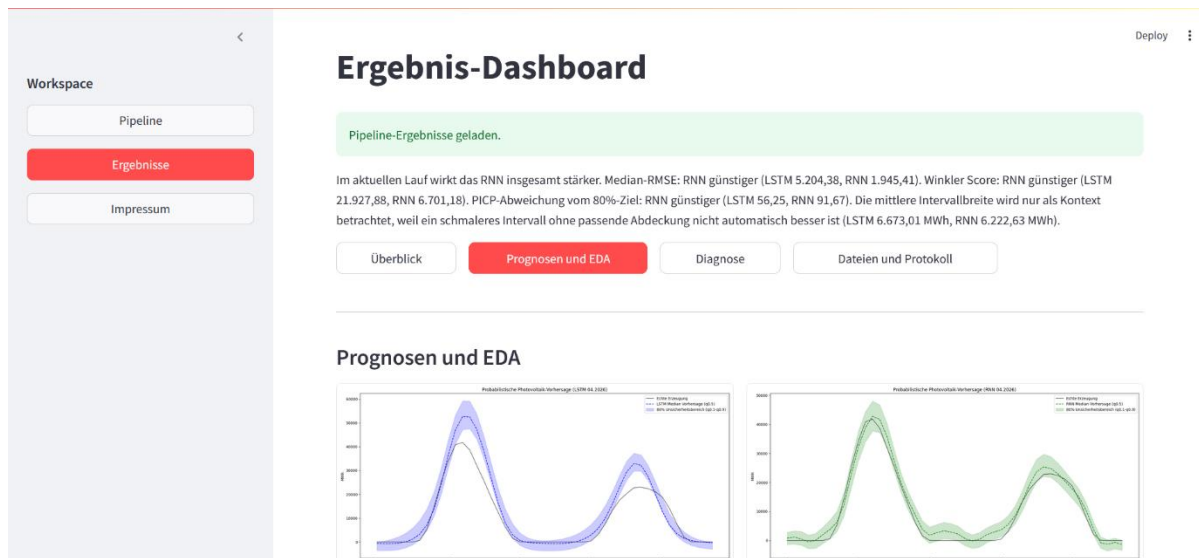


Abbildung 1.4: Ergebnis-Dashboard: Ergebnis-Dashboard in der Rubrik „Prognosen und EDA“ mit allen zugehörigen Grafiken der Prognosen, der Lernkurven und der Ergebnisse der Datenverarbeitung.

1.5.2 Technische Implementierung und Software-Architektur

Das finale Produkt ist eine modularisierte, containerisierte End-to-End-Pipeline, deren Orchestrierung über eine interaktive Streamlit-Webanwendung erfolgt. Die gesamte Architektur ist in eine plattformunabhängige Docker-Laufzeitumgebung eingebettet. Dabei erfolgt eine Datensynchronisation zwischen den Verzeichnissen `/smard_daten/input` und `/smard_daten/output`. Nach dem Upload einer unskalierten SMARD-Rohdaten-CSV (und optionaler Wetter-Features) im Frontend wird die Berechnungs-Pipeline sequentiell ausgeführt. Konsolenausgaben und Fehlermeldungen werden dynamisch in einer Protokolldatei abgefangen und alle generierten Artefakte (validierte CSV-Daten, serialisierte Skalierungs-Metadaten und hochauflösende PNG-Visualisierungen) in das Output-Verzeichnis abgelegt. Die Pipeline gliedert sich wie folgt:

- Explorative Datenanalyse, Bereinigung und Feature Engineering (`eda.py`): In diesem Skript werden die SMARD-Rohdaten geladen, bereinigt und länderspezifische numerische String-Formate über eine Zeitstempel-Analyse isoliert. Neben einer automatisierten Auflösungserkennung wird ein zyklisches Feature Engineering durchgeführt. Die Rhythmen werden über Sinus- und Kosinus-Transformationen der Stunde mathematisch als kontinuierliche Wellenfunktionen operationalisiert. Es existiert eine Schnittstelle (`merge_weather_features`), um optionale Wetterdaten zeitlich exakt mit den Energiedaten zu mergen.

- Chronologische Daten-Transformation und Leakage-freie Sequenzerstellung (`training_utils.py`): Die multivariate Feature-Matrix (bestehend aus PV, Wind Onshore, Erdgas und zyklischen Zeitkomponenten) wird über ein gleitendes Fenster für das überwachte Lernen transformiert. Dabei passt sich die Fenstergröße dynamisch an die erkannte Auflösung an, sodass die Historie immer exakt eine Sequenz von 24 Stunden umfasst. Zur Vermeidung von Data Leakage erfolgt der `MinMaxScaler`-Fit strikt ausschließlich auf dem chronologischen Trainings-Split. Validierungs- und Testdaten werden erst danach transformiert. Insgesamt erfolgt die Datenaufteilung sequentiell im Verhältnis 80 % Train/Validation (davon 10 % Validierung zur Early-Stopping-Steuerung) und 20 % Testdaten. Die Sequenzen werden in ein dreidimensionales PyTorch-Tensorformat der Dimension (B, T, D) überführt. Dabei steht B für die Batch-Größe, T für die Sequenzlänge und D für die Feature-Anzahl.
- Probabilistische rekurrente Modellarchitekturen (`train_lstm.py` und `train_rnn.py`): Im Fokus stehen hier das LSTM und das RNN. Beide Netze verfügen über ein verdecktes Layout mit zwei rekurrenten Schichten, 128 Hidden-Units und einer Dropout-Regularisierung von 20 % zur Vermeidung von Overfitting. Die finale lineare Schicht gibt drei Ausgabeneuronen für die Quantile $q_{0.1}$ (untere Grenze), $q_{0.5}$ (Median) und $q_{0.9}$ (obere Grenze) aus, woraus ein 80 %-Vorhersageintervall entsteht.

Das Training erfolgt nicht-parametrisch über die Minimierung des Pinball Loss (Quantils-Verlustfunktion). Um fachlich unlogische Quantilkreuzungen aktiv zu unterdrücken, setzt sich die Loss-Funktion aus dem Pinball Loss und einer gewichteten *Quantile Crossing Penalty* zusammen. Die Optimierung wird durch den Adam Optimizer mit einer Lernrate von 0.0005 gesteuert, der an einen dynamischen Lernraten-Scheduler gekoppelt ist. Der Early-Stopping-Mechanismus bricht das Training zur Absicherung der Generalisierungsfähigkeit ab, sofern der Validierungsverlust über 25 Epochen stagnierte. Nachdem das Modell mit der Prediction Interval Coverage Probability (PICP) evaluiert wurde, wird die Black-Box-Struktur der Netzwerke über die Permutation Feature Importance aufgebrochen, die den mathematischen Einfluss jedes einzelnen Features direkt auf den Pinball Loss misst.

2 Arbeitspakete

Im Folgenden werden die Arbeitspakete (AP) dargestellt, die in diesem Projekt definiert wurden.

2.1 AP1: Recherche nach geeigneten Datenquellen und explorative Datenanalyse

Bevor entsprechende Architekturen trainiert werden können, müssen zunächst geeignete Datenquellen gefunden werden, die eine fehlerfreie und eine stationäre Datenbasis darstellen. Dabei gilt es folgende Aufgaben zu erledigen:

- Bereinigung der Daten (z. B. Transformation von String-Formaten in numerische Werte)
- Explorative Datenanalyse (EDA) zur Extraktion zyklischer Features
- Analyse der Korrelationen
- Stationaritätsprüfung

Ziel: Generierung einer strukturierten und validierten Eingangsmatrix im Tensorformat und dynamisch generierte EDA-Visualisierungen.

2.2 AP2: Modellarchitektur und probabilistisches Training (LSTM vs. RNN)

Das Ziel besteht im Training der beiden konkurrierenden neuronalen Netze (LSTM/RNN). Hierzu soll eine Trainings-Pipeline konfiguriert werden (Early Stopping, Hyperparameter-Setup), die als Ergebnis die erfolgreich trainierten Modellgewichte für beide Architekturen speichert und automatisiert den Trainingsverlauf protokolliert.

2.3 AP3: Evaluierung, Modellvergleich und Erklärbarkeit

Im Fokus steht ein quantitativer und qualitativer Vergleich der Modellleistungen und die Aufdeckung der Feature-Wichtigkeiten. Um die präzise Architektur zu ermitteln, werden die Verluste (Pinball Loss) auf den Testdaten berechnet und für das LSTM und RNN gegenübergestellt. Durch Implementierung der Permutation Feature Importance soll der mathematische Einfluss

der einzelnen Eingangsvariablen auf die Vorhersage charakterisiert werden. Schlussendlich sollen die finalen Prognosedigramme, die das reale Einspeiseprofil inklusive des prognostizierten, dynamischen Unsicherheitsbandes abbilden, generiert werden.

2.4 AP4: Containerisierung und Projektdokumentation

Die finale Pipeline soll in eine plattformunabhängige Laufzeitumgebung überführt werden. Dabei soll eine fehlerfreie Datensynchronisation sichergestellt werden. Im Fokus steht eine vollständig reproduzierbare, containerbasierte Anwendung mit umfassender Projektdokumentation.

3 Vorgehensweise und Projektumsetzung

3.1 Kick-Off

Im Rahmen des Kick-Offs wurde der Projektumfang, die Meilensteine und die Kommunikation im Team definiert.

Definition des Projektumfangs: Konzeption, Implementierung und containerisierte Bereitstellung einer Pipeline zur probabilistischen Zeitreihenvorhersage der Energieeinspeisung auf Basis historischer Zeitreihendaten. Dabei sollen die unskalierten Rohdaten automatisiert bereinigt, um zyklische sowie meteorologische Merkmale erweitert und in ein geeignetes Tensorformat überführt werden. Der Fokus liegt auf dem modellseitigen Training und einem empirischen Leistungsvergleich zweier Architekturen (LSTM und RNN), um statt starrer Punktprognosen dynamische 80 % Konfidenzintervalle zu generieren.

Meilensteine: Folgende Meilensteine wurden im Rahmen dieses Projektes definiert:

- Entwicklung eines Konzeptes zur Datenbereinigung und explorativen Datenanalyse
- Entwicklung zweier containerbasierten Architekturen LSTM/RNN
- Vergleich beider Architekturen mit Zeitreihendaten unterschiedlicher Länge (unterschiedlicher Zeiträume) und Auflösung (z. B. viertelstündige oder stündliche Auflösung)

Kommunikationsstrukturen: Für ein effizientes und agiles Vorgehen im Team wurden feste Kommunikationsstrukturen beschlossen. Die Koordination und Fortschrittsbesprechung basiert auf wöchentlichen Sitzungen, in denen der aktuelle Fortschritt besprochen, Aufgaben verteilt und strategische Entscheidungen getroffen werden. Darüber hinaus dienen die wöchentlichen Sitzungen den Gruppenmitgliedern zum Einholen von Feedback nach erledigten Teilaufgaben.

3.2 Zeitplan

Die gesamte Projektlaufzeit wurde für insgesamt elf Wochen geplant, wobei der Fokus auf einer agilen Entwicklungsumgebung lag. Dabei wird das Produkt durch die hier getätigte Zusammenstellung der Arbeitspakete inkrementell optimiert:

- Die Bearbeitung des AP1 wurde für KW 17 bis 19 geplant und beinhaltet die Definition des Projektes, die Recherche nach geeigneten Daten und die EDA.
- Zur Bearbeitung von AP2 und AP4 wurde KW 20 bis 22 angesetzt und beinhaltet neben dem Erstellen der LSTM/RNN Architekturen auch die ersten Testläufe.
- Für KW 23 bis 25 wurde die Bearbeitung des AP3 angesetzt. Hier liegt der Fokus auf dem qualitativen und quantitativen Vergleich beider Architekturen mit Datensätzen unterschiedlicher Zeiträume und Auflösung (z. B. Viertelstunde vs. Stunde).

KW/Ereignisse	17	18	19	20	21	22	23	24	25	26	27
Kick-Off und AP1 Erste Ideensammlung Definition der Projektziele Recherche nach geeigneten Daten Explorative Datenanalyse											
AP2 und AP4 Erstellen eines LSTM und RNN Training des RNN und LSTM Erste Testläufe Containerisierung											
AP3 Quantitativer und qualitativer Vergleich Testen unterschiedlicher Datensätze · kurze vs. lange Zeiträume · Variation der Datenauflösung											
AP4 Behebung letzter Fehler Projektdokumentation											

Jeder der oben im Gantt-Chart präsentierten Balken entspricht einer Iteration, wobei in jeder Iteration die Bearbeitung verschiedener APs im Vordergrund stehen. Durch das iterative Vorgehen wird das bestehende Produkt inkrementell verbessert. Nach jeder Iteration stehen feste teaminterne Feedbackschleifen im Vordergrund, die die Basis darstellen, um in die nächste Iteration zu starten.

3.3 Organisation im Projektteam

Alle Programmierarbeiten werden in Python durchgeführt. Für die praktischen Arbeiten wird ein [GitHub](#) Repository angelegt ([Probabilistische-Energiedatenvorhersage](#)). Die gesamte Anwendung wird in Docker containerisiert und bereitgestellt.

Zunächst werden alle Programmierarbeiten lokal auf einem Computer durchgeführt und anschließend per `git push` Befehl mit dem [GitHub Repository](#) synchronisiert.

Die Bearbeitung des Projektes erfolgt in zeitlich getakteten Iterationen mit einer Länge von drei Wochen (siehe Gantt-Chart Kapitel 3.2). Schwerpunkt in den jeweiligen Iterationen sind verschiedene APs. Dabei sollen vor Beginn einer Iteration folgende Punkte im Team vereinbart werden:

- Was ist wertvoll für das Produkt?
- Was soll erreicht werden?
- Wie soll es gemacht werden?

Nach Abschluss einer Iteration werden die Ergebnisse intern im Team evaluiert. Dies dient der gemeinsamen Diskussion und der Planung der nächsten Arbeitsschritte. Das so erhaltene Feedback bildet wiederum die Basis für die nächste Iteration und zur weiteren Konkretisierung des Zeitplans (siehe Kapitel 3.2).

Weiterhin sollen gemeinsame, teaminterne Retrospektiven am Anfang einer neuen Iteration stattfinden. Dabei stehen die folgenden Punkte im Vordergrund:

- Was lief beim letzten Mal gut?
- Was lief weniger gut und sollte verändert werden?
- Wer kümmert sich um die Veränderungen und die nächsten Schritte?

3.4 Umsetzung der Projektarbeit

Das gesamte Projekt konnte in dem dafür vorgesehenen Zeitplan umgesetzt werden. Dabei erwies sich die im Zeitplan getätigte Aufteilung der Umsetzung als zweckmäßig.

Ergebnisse der Iteration 1 (Zeitraum: KW 17 bis 19)

Als Datenquelle wurde [SMARD Marktdaten](#) der Bundesnetzagentur verwendet, die die benötigten Daten unter der Lizenz CC BY 4.0 zur Verfügung stellen. Es wurde eine Python Skript für die EDA erstellt, um die Daten zu bereinigen, zyklische Features zu extrahieren, die Korrelationen zu analysieren und eine Stationaritätsprüfung durchzuführen. Die Herausforderung der Programmierarbeit bestand darin, dass das Skript so dynamisch angepasst werden musste, dass die EDA mit den unterschiedlichen zeitlichen Auflösungen der Daten problemlos umgehen kann. Dazu wurde eine dynamische Auflösungserkennung und eine Zeitraumermittlung eingefügt, die den Abstand zwischen den ersten beiden Zeilen in Minuten berechnet. Diese Dynamik erwies sich als zweckführend, um mit den verschiedenen zeitlichen Auflösungen umzugehen, die SMARD Marktdaten zur Verfügung stellt.

Ergebnisse der Iteration 2 (Zeitraum: KW 20 bis 22)

Sowohl das LSTM als auch das RNN wurden als hochgradig modularisiertes Framework zur Verarbeitung, zum Training, zur Evaluierung und zur Erklärbarkeit der probabilistischen Modelle ausgestattet. Dabei wurden in die Modelle folgende Bausteine integriert:

- Wind Onshore und Erdgas werden dynamisch über Textsuche ermittelt und als exogene Zeitreihen-Features bereitgestellt.
- Beide Architekturen enthalten zyklische Zeit-Transformationen:
 - `hour_sin/hour_cos` (Tagesrhythmus im 24 Stunden-Zyklus).
 - `weekday_sin/weekday_cos` (Wochenrhythmus im 7 Tage-Zyklus).
 - `is_weekend` (binäres Flag 0 oder 1 zur Erfassung von Wochenend-Effekten im Stromnetz).
- Dynamisches Wetter-Modul als optionale Erweiterung: durchsucht Wetter CSVs nach definierten Begriffen, bereinigt deutsche Zahlenformate (Komma zu Punkt) und interpoliert fehlende Werte zeitlich.
- Die Daten werden streng chronologisch aufgeteilt: 80 % Training/Validierung und 20 % Testdaten. Der Trainingsbereich wird intern weiter in ein echtes Trainings-Set und ein Validierungs-Set (10 %) für den Early Stopping-Mechanismus unterteilt.
- Saubere Skalierung (MinMaxScaler): Die Scaler werden ausschließlich auf den chronologischen Trainingsdaten gefittet, um jegliches Data-Leakage aus der Zukunft (Testset) zu verhindern.

- Die Sequenzlänge wird automatisch an die Zeitauflösung der CSV angepasst. Das Modell schaut immer 24 Stunden in die Vergangenheit (z. B. 96 Zeitschritte bei einem 15-Minuten-Raster).
- Pinball Loss (Quantils-Regression): Schätzt insgesamt drei Quantile.
- Quantile Crossing Penalty: Das Modell wird bestraft, falls sich Vorhersagen der Quantile überschneiden.
- Quantil-Sortierung: Die Reihenfolge der Quantile $q_{0.1} \leq q_{0.5} \leq q_{0.9}$ wird strikt eingehalten.
- Optimizer: Das Modell nutzt den Adam-Optimizer.
- Lernrate wird dynamisch angepasst: Fällt der Validierungsverlust sieben Epochen lang nicht, wird die Lernrate halbiert.
- Early Stopping: Das Training wird direkt abgebrochen, wenn sich der Validierungsverlust über 25 Epochen nicht verbessert hat. Dabei wird der beste Modell-Checkpoint wieder hergestellt.
- Prediction Internal Coverage Propability (PICP): Prozentsatz der Werte, die tatsächlich innerhalb des 80 % Konfidenzintervalls liegen.
- Mean Internal Width (MWh): Die durchschnittliche Breite des Unsicherheitsbandes in MWh.
- Median RMSE (MWh): Der Root Mean Squared Error für die Median-Prognose ($q_{0.5}$).
- Winkler Score (80 % MWh): Schmale Intervalle werden belohnt und Ausreißer außerhalb des Bandes werden mathematisch bestraft.
- Quantil-Kalibrierung: Die beobachtete Abdeckung wird mit der theoretisch erwarteten Abdeckung für alle Quantile verglichen.
- Permutation Feature Importance: Hier wird jedes Eingangs-Feature auf den Testdaten durcheinander gewürfelt und der Anstieg des Pinball Loss gemessen. Dabei wird mathematisch gezeigt, welche Variable (z. B. Uhrzeit, Wind, Erdgas) den jeweils größten Einfluss auf die Vorhersage haben.
- Automatische Sicherung der Artefakte:
 - Visualisierungen (PNG): Lernkurve, Vorhersageplot mit 80 % Band, Quantil-Kalibrierungsdiagramm, Feature Importance Balkendiagramm.
 - Tabellarische Daten (CSV): Detaillierte Zeitreihen-Prognosen, Metriken-Übersichten, Trainings-Historie und Permutation-Importances.

- Serialisierte Modelle und Metadaten (JSON/PTH/PKL): Es werden die PyTorch-Gewichte (.pth), die gefitteten Scaler (.pkl) und eine vollständige JSON-Metadatei mit allen Hyperparametern und Split-Details gespeichert.

Zur Gewährung einer plattformunabhängigen Ausführbarkeit, vollständigen Reproduzierbarkeit und einfachen Skalierung wurde die gesamte Datenpipeline containerisiert. Die technische Kapselung basiert hierbei auf Docker und wird über Docker Compose deklarativ orchestriert.

Beim Build-Prozess werden alle Python-spezifischen Bibliotheken (u. a. PyTorch, Streamlit, scikit-learn und pandas) deterministisch über eine fixierte `requirements.txt` installiert.

Architektur und Laufzeitumgebung: Die Laufzeitumgebung arbeitet über eine Zwei-Wege-Volumensynchronisation: Eingabedateien werden in `/app/data/input` empfangen. Alle Trainingsergebnisse, Modelle und Grafiken werden nach `/app/data/output` verschoben.

Ergebnisse der Iteration 3 (Zeitraum: KW 23 bis 25)

Zur Validierung der Robustheit wurde die implementierte Datenpipeline mit heterogenen CSV-Datensätzen unterschiedlicher zeitlicher Granularität evaluiert. Im Rahmen dieser Testläufe konnten initiale Fehlerquellen im Quellcode systematisch identifiziert und behoben werden, sodass eine durchgehend stabile Ausführung gewährleistet ist. Die zentralen Erkenntnisse und Performanzmetriken dieser Evaluierung werden detailliert in Kapitel 4 diskutiert.

Ergebnisse der Iteration 4 (Zeitraum: KW 26 bis 27)

Die vierte Iteration wurde genutzt, um die Architektur und Laufzeitumgebung zu optimieren. Hierfür wurden folgende Eigenschaften implementiert und optimiert:

Hauptseiten und Navigationsstruktur: Die Anwendung gliedert sich insgesamt in drei Hauptbereiche:

- Pipeline-Seite: Upload-Interface für SMARD-Rohdaten und optionale Wetter-CSVs
- Ergebnis-Dashboard: Detaillierte visuelle und tabellarische Auswertung der trainierten Modelle.
- Impressum und Projekterklärung: Vollständige fachliche Dokumentation des Projektes samt Erklärung von Methodik, Metriken, Datenherkunft und Grenzen.

Modulare Dashboard-Tabs: Das Ergebnis Dashboard wurde in vier spezialisierte Registerkarten unterteilt:

- Tab 1: Überblick: Direkter Modellvergleich, Metriken-Karten, Trainingsumfang/Metadaten.
- Tab 2: Prognosen und EDA: Modellprognosen (Gegenüberstellung der 80 %-Konfidenz-band-Plots und Verlust-Lernkurven für LSTM und RNN) und EDA.
- Tab 3: Diagnose und Erklärbarkeit: Kalibrierungsdiagramme und Permutation Feature Importance.
- Tab 4: Dateien und Protokoll: Dateimanager (tabellarische Übersicht aller erzeugten Artefakte mit Dateigröße und direktem CSV-Download) und Log-Viewer (Einsicht in die letzten Zeichen des Ausführungsprotokolls).

Da die Frist der Projektabgabe bis zum Beginn der Vorlesungszeit im nächsten Semester verschoben wurde (KW40), wurde die verbleibende Zeit genutzt, um den Projektbericht fertigzustellen und das GitHub Repository zu finalisieren.

4 Ergebnisse und Diskussion

4.1 Explorative Datenanalyse

Die unskalierten Rohdaten in Abbildung 4.1 zeigen den gesamten kontinuierlichen Zeitverlauf vom 09. April 2026 bis zum 19. April 2026 im 15-Minuten Takt. Die Photovoltaik (PV)-Erzeugung besteht aus einzelnen, scharfen Ausschlägern (Tagespeaks), während die Windenergie eine wellenartige Struktur zeigt.

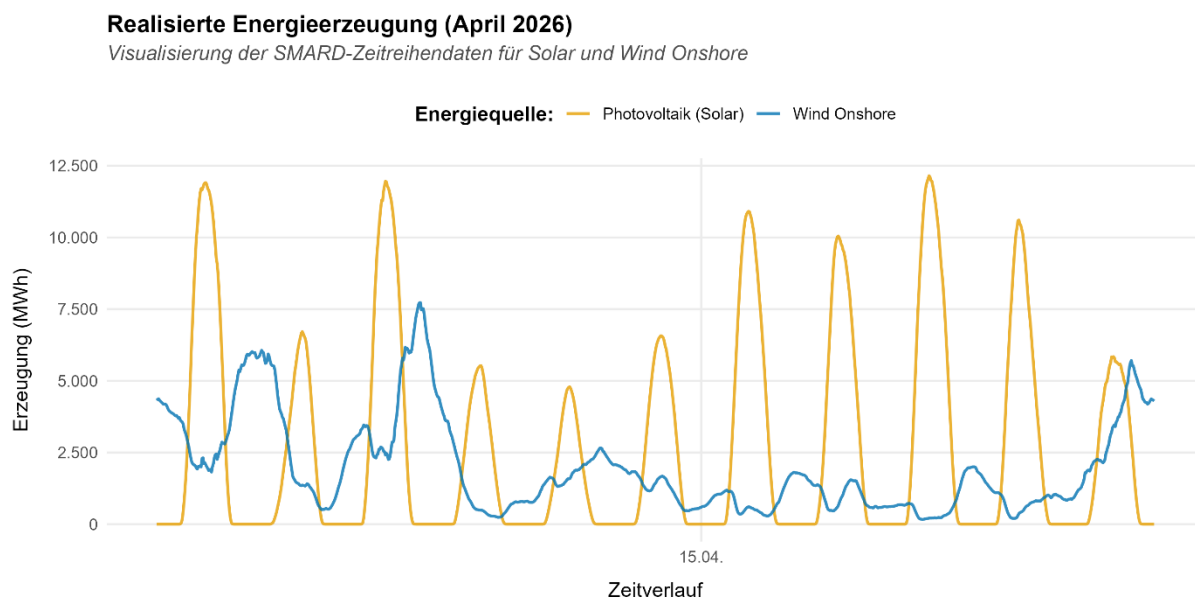


Abbildung 4.1: Ergebnisse der EDA – Energieerzeugung: Solar- und Wind Onshore-Energieerzeugung über elf Tage (Rohdaten) vom 09.04.2026 bis zum 19.04.2026.

Die unskalierten Rohdaten zeigen einen kontinuierlichen Zeitverlauf. Die PV-Erzeugung besteht aus solaren Tagespeaks, während die landbasierte Windenergie einen stochastischen, wellenartigen Verlauf aufweist.

4.1.1 Datensatzstruktur und Vollständigkeit

Die Überprüfung der Struktur des [SMARD-Datensatzes](#) zeigt eine lückenlose Zeitreihe von insgesamt 1056 Beobachtungen im Zeitraum vom 09. April 2026 bis zum 19. April 2026.^[5] Da der Datensatz in einer viertelstündigen Auflösung vorliegt, lässt sich der Datenumfang mathematisch validieren:

- 96 Beobachtungen am Tag (24 Stunden x 4 Intervalle/Stunde)

- Elf Tage Beobachtungszeitraum (bzw. zehn volle Tage plus Randstunden nach der Bereinigung)

Die insgesamt 19 Spalten (Features) weisen keine fehlenden Werte auf, was eine stabile Grundlage für das Machine Learning bietet.

4.1.2 Deskriptive Analyse der Kernkomponenten

Die statistischen Kennzahlen der primären volatilen Energieträger offenbaren fundamentale Unterschiede in der jeweiligen physikalischen Verteilungscharakteristik, siehe Abbildung 4.2:

- PV: Der Mittelwert der solaren Einspeisung liegt bei 2056.23 MWh mit einer maximalen Peak-Leistung von 12070.45 MWh. Die Standardabweichung von 3353.81 MWh übersteigt den Mittelwert, was auf inhärente Volatilität der Solarenergie hindeutet. Das 25 %-Quantil liegt bei exakt 0.00 MWh und das 50 %-Quantil bei 0.01 MWh. Statistisch gesehen wird in über 50 % des gesamten Beobachtungszeitraums faktisch keine solare Einspeisung beobachtet, was eine direkte Folge der Nachtstunden ist. Herkömmliche Modelle neigen dazu die Spitzenwerte drastisch zu unterschätzen. Dies rechtfertigt den in dieser Arbeit gewählten probabilistischen Quantils-Ansatz (Pinball Loss).
- Wind Onshore: Die landbasierte Windkraft zeigt einen Mittelwert von 1562.65 MWh und ein Maximum Wert von 7729.33 MWh. Das 25 %-Quantil liegt bei 286.38 MWh. Im Gegensatz zur PV bricht die Windkraftproduktion nachts nicht systematisch ein und stellt somit einen kontinuierlichen stochastischen Prozess dar, der unabhängig von den astronomischen Tageszeiten operiert.

Wird das aggregierte, durchschnittliche Tagesprofil in Abbildung 4.2 betrachtet, treten diese Charakteristiken deutlich hervor. Die blaue PV-Kurve zeigt die klassische Glockenform und beginnt exakt bei 6:00 Uhr morgens zu steigen und erreicht ihr Maximum um 12:00 bis 13:00 Uhr während des höchsten Sonnenstandes. Bis 20:00 Uhr fällt die Kurve wieder auf die Nulllinie ab. Zwischen 20:00 und 6:00 Uhr verläuft die Kurve exakt auf der Nulllinie, da die Sonne in der Nacht nicht scheint. Im aggregierten Mittel verläuft die Windkraft relativ konstant in einem Band zwischen 1300 und 2400 MWh. Es gibt hier im Gegensatz zur Solarenergieproduktion keinen festen Tag-/Nacht-Rhythmus.

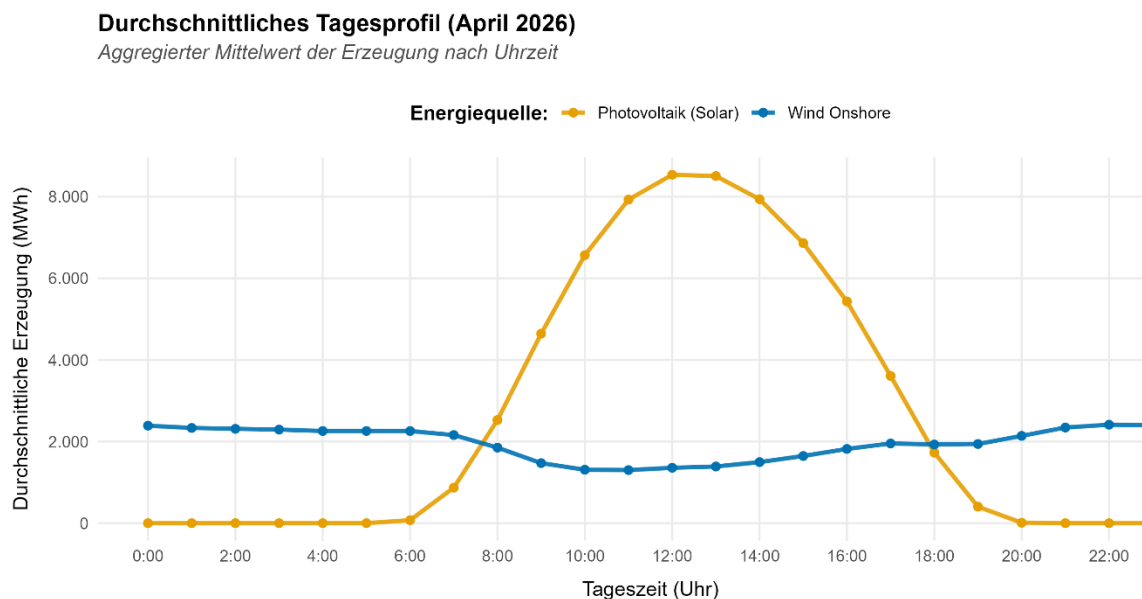


Abbildung 4.2: Durchschnittliches Tagesprofil: Durchschnittliches Tagesprofil der PV und Wind Onshore Energieerzeugung aggregiert über elf Tage.

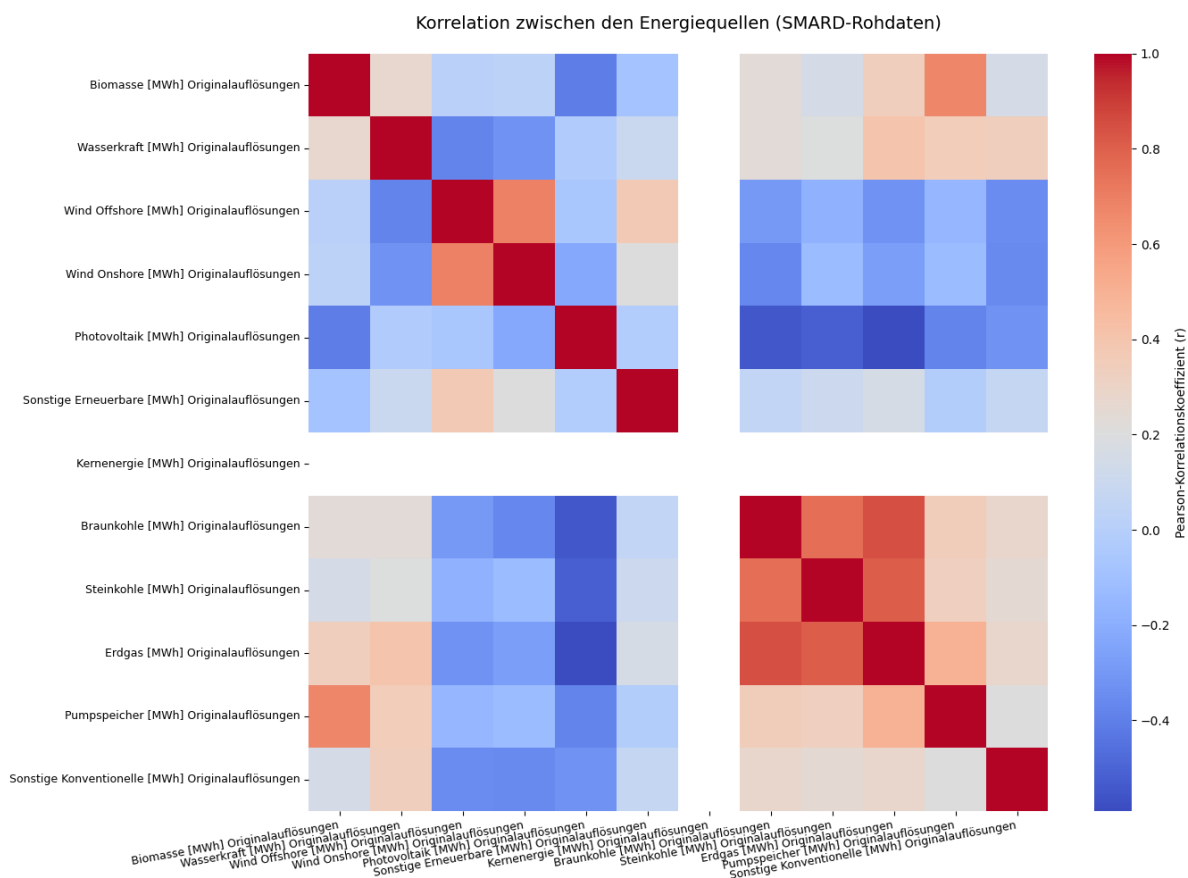


Abbildung 4.3: Korrelationsmatrix: Korrelationen zwischen den Energiequellen dargestellt in einer Heatmap.

Die Korrelationsmatrix (Heatmap) in Abbildung 4.3 zeigt die linearen Korrelationskoeffizienten nach Pearson zwischen allen im Datensatz enthaltenen Erzeugungsquellen. Zwischen der PV und der Onshore-Windkraft zeigt sich eine schwach negative Korrelation (etwa -0.15). Diese Beobachtung deckt sich mit der meteorologischen Realität, da stark windprägende Tiefdruckgebiete häufig mit einer erhöhten Wolkenbedeckung einhergehen, die die solare Einstrahlung drastisch dämpft.

Da die Kernkraftwerke in dem betrachteten Zeitraum des Jahres 2026 stillgelegt sind, weist dieses Feature eine Varianz von 0 auf. Mathematisch führt dies bei der Berechnung des Pearson-Koeffizienten zu nicht definierten Ausdrücken. Daher bleibt die entsprechende Zeile und Spalte visuell leer.

4.1.3 Mathematische Stationaritätsprüfung

Vor dem Training der Modelle muss sichergestellt werden, dass die Zeitreihe stationär ist, da stochastische Trends zu einer Scheinkorrelation und instabilen Gradientenstrukturen führen können. Im Rahmen dieser Arbeit wurde der Augmented Dickey-Fuller-Test (ADF-Test) auf die PV-Erzeugungsdaten angewendet. Dabei liefert die Teststatistik einen empirischen Wert von -6.5474. Dieser Wert liegt unter dem kritischen Schwellenwert, wodurch ein asymptotischer p -Wert von 0.0000 resultiert. Somit kann bei einem Signifikanzniveau von $\alpha = 1\%$ ausgesagt werden, dass die Zeitreihe im statistischen Sinne stationär ist.

4.1.4 Tensor-Transformation für das überwachte Lernen

Für das sequentielle Training des LSTMs und RNNs wurden die bereinigten Daten über ein gleitendes Fenster in ein dreidimensionales Tensorformat überführt. Es wurde eine Fenstergröße von 96 Zeitschritten definiert, was exakt der Periode eines vollen Tages (24 Stunden $\times$ 4 Intervalle) entspricht.

Bedingt durch diese Transformation verringert sich die Anzahl der effektiv nutzbaren Datenpunkte mathematisch bedingt von 1056 auf 960 Sequenzdaten.

- Somit besitzt die Feature-Matrix X die Dimensionen $(960, 96, D)$, wobei D für die Anzahl der Eingangsmerkmale steht. Das Modell liest somit 960-mal ein Fenster aus 96 aufeinanderfolgenden Viertelstunden ein.

- Der Zielvektor y besitzt die Dimension (960,) und repräsentiert den drauffolgenden 97. Zeitschritt.

Es deckt sich mit der physikalischen Realität, dass der erste zu erlernende Wert der Matrix exakt 0.0 MWh beträgt, da nach den ersten 96 Zeitschritten (Tag 1) die Prädiktion für den Folgetag exakt um Mitternacht 0:00 Uhr startet. Dies ist der Zeitpunkt, an dem die solare Erzeugung zwingend null betragen muss. Das verifiziert die logische und chronologische Integrität der gesamten Datenpipeline.

4.2 Qualitativer und quantitativer Vergleich der Architekturen

Das Modelltraining wurde mit drei Datensätzen unterschiedlicher Auflösung durchgeführt:

- Zeitraum 09.04. bis 19.04.2026 in 15-minütiger Auflösung (April 2026)
- Zeitraum 09.04. bis 19.04.2026 in 60-minütiger Auflösung (April 2026)
- Zeitraum 01.03. bis 19.04.2026 in 15-minütiger Auflösung (März-April 2026)

Die Lernkurven beider Architekturen (LSTM – rechts und RNN – links) sind in Abbildung 4.4 gezeigt:

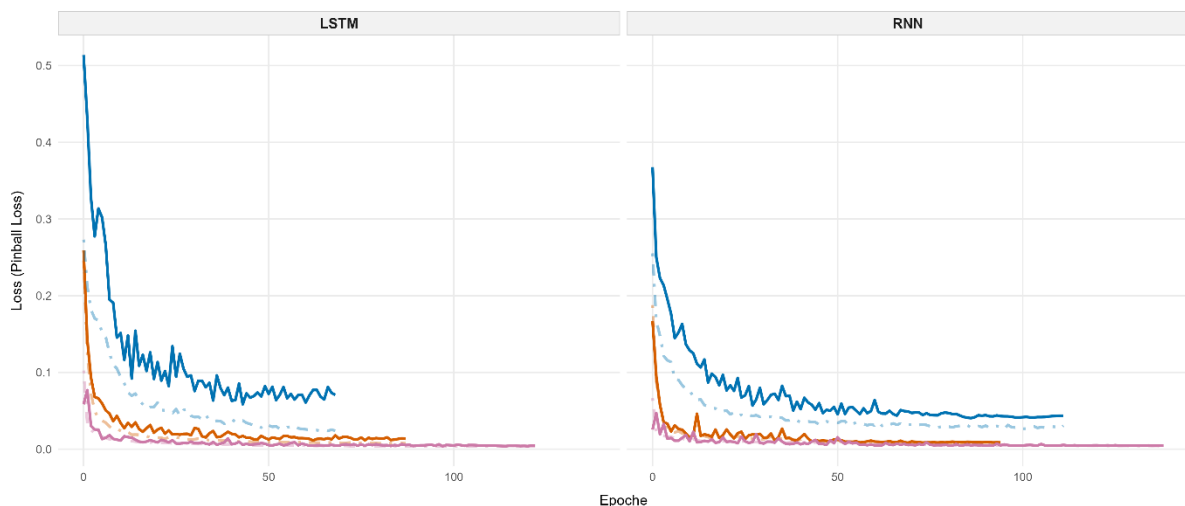


Abbildung 4.4: Lernkurven beider Architekturen (LSTM und RNN) im direkten Vergleich. Rot – 15-minütige Auflösung (April 2026), pink - 15-minütige Auflösung (März-April 2026) und blau – 60-minütige Auflösung (April 2026). Die durchgezogenen Kurven entsprechen der Validierung und die gestrichelten Kurven entsprechen dem Training.

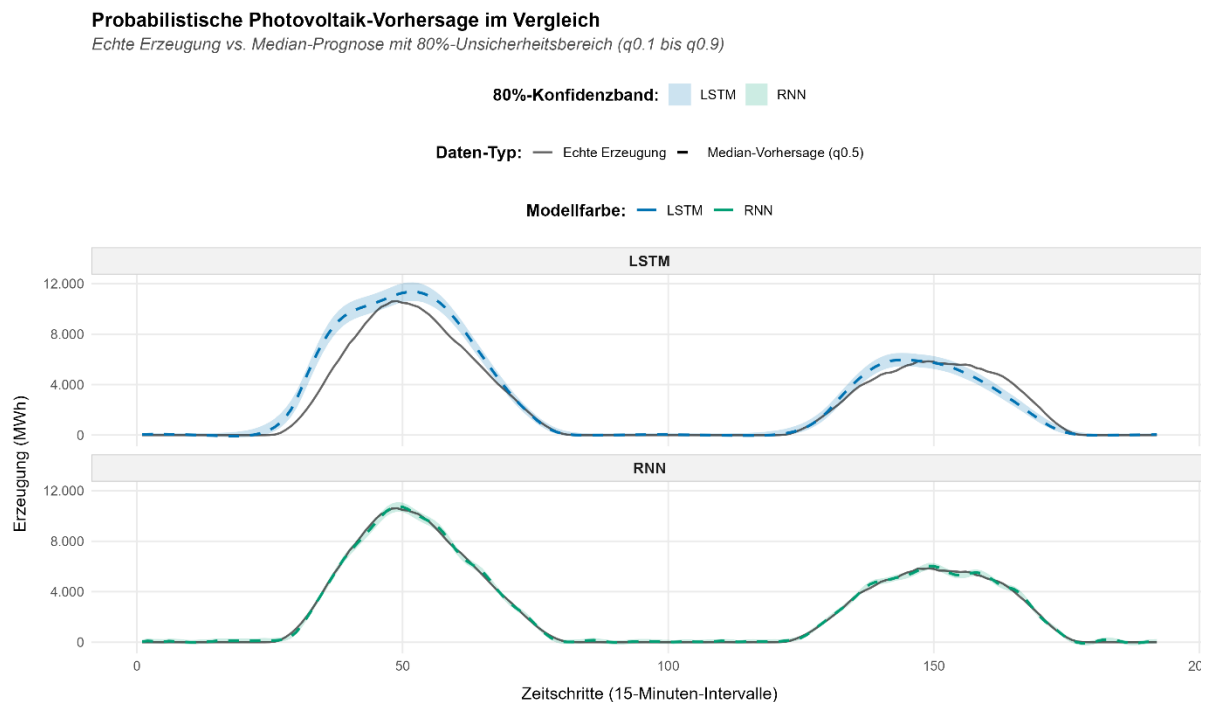


Abbildung 4.5: PV-Vorhersage (1): Probabilistische PV-Vorhersage im direkten Architekturvergleich (Datensatz April 2026, 15-minütige Auflösung).

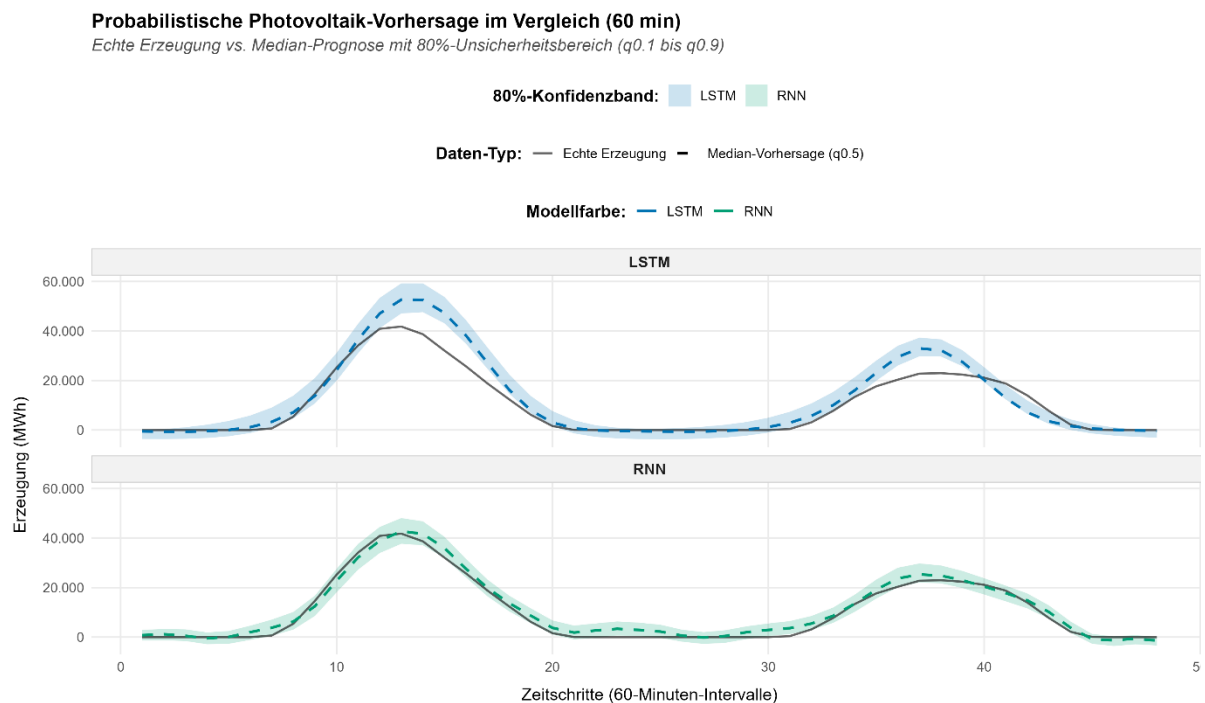


Abbildung 4.6: PV-Vorhersage (2): Probabilistische PV-Vorhersage im direkten Architekturvergleich (Datensatz April 2026, 60-minütige Auflösung).

Die Vorhersagen und 80 %igen Konfidenzbänder im direkten Architekturvergleich des April 2026 Datensatzes mit 15-minütiger Auflösung ist in Abbildung 4.5 und mit 60-minütiger Auflösung in Abbildung 4.6 gezeigt. Vorhersagen und Konfidenzbänder des März-April 2026 Datensatzes sind in Abbildung 4.7 dargestellt.

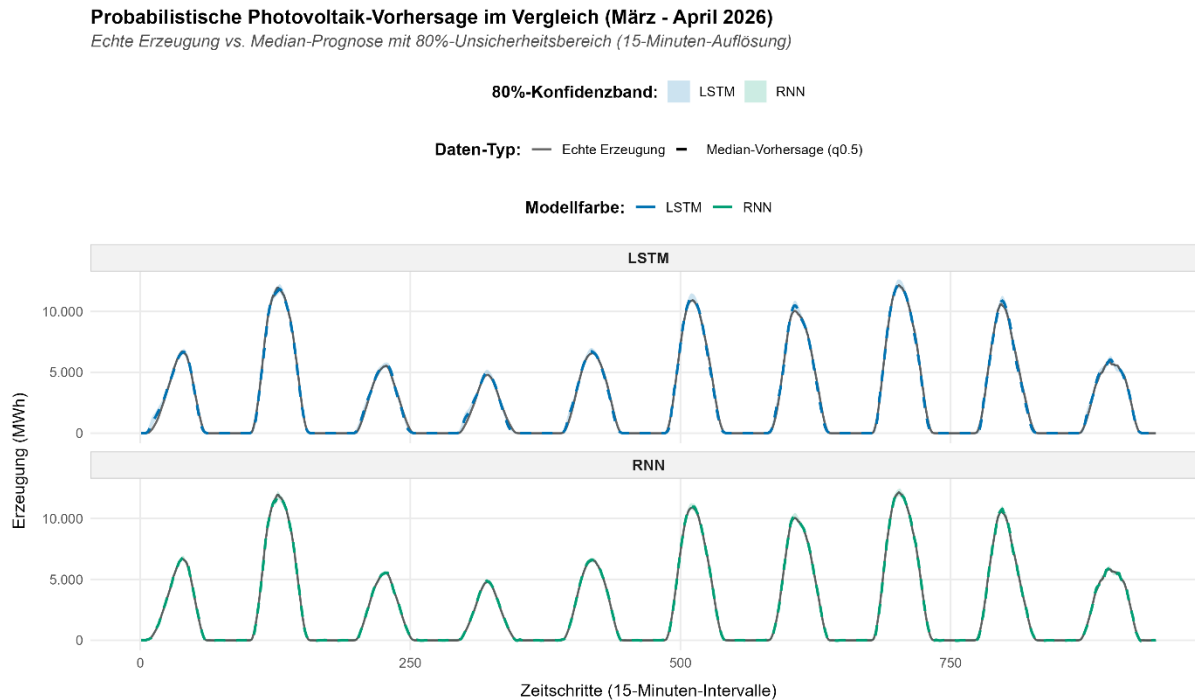


Abbildung 4.7: PV-Vorhersage (3): Probabilistische PV-Vorhersage im direkten Architekturvergleich (Datensatz März-April 2026, 15-minütige Auflösung).

Die kombinierten Lernkurven zeigen den Verlauf des Pinball-Loss über die Epochen, wobei nach den Architekturen LSTM (Abbildung 4.4 links) und RNN (Abbildung 4.4 rechts) getrennt wurde. In beiden Modellen zeigt sich ein stabiles Konvergenzverhalten, das durch einen integrierten Early-Stopping-Mechanismus vorzeitig abgebrochen wurde, nachdem der Validierungsverlust stagnierte.

Die Auswertung der Vorhersagediagramme in den Abbildungen 4.4 bis 4.7 zeigen den präzisen solaren Glockenverlauf und wie sich die Unsicherheitsbänder (80 % Konfidenzintervalle) verhalten. Im April 2026 Datensatz (15-minütige Auflösung) treffen sowohl LSTM als auch RNN den realen Verlauf (schwarze Linie), siehe Abbildung 4.5. Das hellblaue (LSTM) bzw. hellgrüne (RNN) Band schmiegt sich extrem an die Kurven an. Insgesamt weisen die Modelle bei dieser hohen Auflösung und klarem Wetter eine sehr hohe Vorhersagesicherheit auf. Nachts verengt sich dieses Band auf die Nulllinie. Somit haben die Modelle gelernt, dass ohne Sonnenlicht

keine solare Varianz existieren kann. Der April 2026 Datensatz (60-minütige Auflösung) zeigt, dass das LSTM am ersten Tag die absolute Spitze leicht überschätzt, während das RNN diese Spitze präzise trifft, siehe Abbildung 4.6. Da sich durch die stündliche Aggregation die mathematische Unsicherheit des Modells pro Zeitschritt erhöht, führt dies dazu, dass das Unsicherheitsband signifikant breiter ist als im 15-Minuten-Szenario. Der März-April 2026 Datensatz (15-minütige Auflösung) weist über den längeren Testzeitraum die höchste Robustheit auf. Der Median schmiegt sich fehlerfrei an die unterschiedlichen Peak-Höhen der Testtage an. Bedingt durch den längeren Zeitraum und die größere Datenmenge, ist das Unsicherheitsband extrem scharf und schmal kalibriert. Das Modell hat durch die erhöhte Datenmenge eine so hohe statistische Sicherheit gewonnen, dass der Risikokorridor minimiert werden konnte. Die PICP bleibt jedoch hoch.

Insgesamt liefern die Modelle keine starren Werte, sondern einen dynamischen Sicherheitskorridor. Je feiner die Auflösung und je größer der Datensatz, desto mehr schärft sich das Konfidenzband. Genau diese Dynamik liefert dem Netzbetreiber den gewünschten Mehrwert: an der Breite des Konfidenzbandes kann sofort das mathematische Risiko für den Handelseinsatz abgeschätzt werden.

Tabelle 4.1: Güteparametervergleich: Vergleich der Güteparameter aus den drei Datensätzen.

Parameter	Datensatz April 2026, 15-minütige Auflösung	Datensatz April 2026, 60-minütige Auflösung	Datensatz März-April 2026, 15-minütige Auflösung
LSTM-PICP	62.50 %	56.25 %	82.23 %
LSTM-RMSE	935.94	5204	154.21
LSTM-Breite	696.60	6673	291.60
LSTM-Crossing	0.00 %	0.00 %	0.00 %
RNN-PICP	92.19 %	91.67 %	97.55 %
RNN-RMSE	126.23	1945	56.83
RNN-Breite	409.91	6222	233.33
RNN-Crossing	0.00 %	0.00 %	0.00 %
Train	692	173	3384
Validierung	76	19	376
Test	192	48	940
Fenster	96	24	96

Im direkten Vergleich der Modellarchitekturen (Tabelle 4.1) erzielt das RNN konsistent bessere Punktprognosen und breitere, konservativere Intervallabdeckungen. Während das LSTM im kurzen 15-Minuten-Datensatz einen RMSE von 935.94 MWh aufweist, liegt der RMSE des

RNN mit 126.23 MWh um ein Vielfaches niedriger. Das RNN erreicht hinsichtlich der Intervallabdeckung im umfangreichen Datensatz (März-April 2026) einen PICP-Wert von 97.55 %, während das LSTM mit 82.23 % sehr nahe am nominellen Zielwert des 80 %-Konfidenzintervalls liegt.

Mit Vergrößerung des Schritintervalls von 15 auf 60 Minuten nimmt die Modellunsicherheit deutlich zu. Dabei steigt der Median-RMSE des LSTMs von 935.94 auf 5204 MWh, wobei sich die mittlere Intervallbreite von 696.60 auf 6673 MWh ungefähr verzehnfacht. Diese Beobachtung lässt sich dadurch begründen, dass pro Zeitschritt ein wesentlich größerer Energiebetrag aggregiert wird und flüchtige Zwischenschwankungen verloren gehen.

Die Vervielfachung des Trainingsumfangs von 692 auf 3384 Sequenzen im März-April Datensatz führt zu einer signifikanten Verschärfung und Präzisierung der Vorhersageintervalle. Dabei sinkt die mittlere Intervallbreite von 696.60 auf 291.60 MWh, während der PICP-Wert von 62.50 auf 82.23 % ansteigt. Damit wird die angestrebte 80 % Abdeckung des Konfidenzintervalls nahezu perfekt getroffen. Beim RNN verringert sich die Intervallbreite von 409.91 auf 233.33 MWh bei einer hohen Abdeckungsrate von 97.55 %. Durch das zusätzliche Datenvolumen wird den Modellen somit ein deutlich schärferes Unsicherheitsband bei gleichzeitig hoher Verlässlichkeit ermöglicht.

Sowohl das LSTM als auch das RNN weisen eine Quantile Crossing Rate von 0.00 % auf. Das bedeutet, dass die mathematische Reihenfolge der ausgegebenen Quantile ($q_{0.1} \leq q_{0.5} \leq q_{0.9}$) eingehalten wird und fachlich inkonsistente Überkreuzungen vollständig unterdrückt werden.

5 Fazit und Reflexion

Im Rahmen dieses Projektes wurde eine Docker-basierte Laufzeitumgebung bereitgestellt, mit der SMARD-Zeitreihendaten zur probabilistischen PV-Einspeisungsprognose verarbeitet werden können. Mittels Verknüpfung von rekurrenten Schichten mit einem nicht-parametrischen Pinball Loss konnte ein dynamischer Risikokorridor konstruiert werden, der statt starrer Punktprognosen einen verlässlichen Sicherheitskorridor liefert. Insgesamt erfüllt das Projekt alle Anforderungen der AP.

Das finale Produkt stellt ein kostenloses Tool dar, das es Netzbetreibern ermöglicht vorab zu planen, um den Bedarf an kostenintensiver Ausgleichsenergie zu minimieren. Durch die explizite Visualisierung der Unsicherheit sind Netzbetreiber in der Lage die Unsicherheit in die jeweiligen Kalkulationen einzubeziehen. Daher erfüllt das Projekt seinen Nutzen und eignet sich nun für erste Testläufe einer realen Nutzerschaft durch Netzbetreiber.

Die empirischen Ergebnisse erlauben folgende Kernkomponenten der Untersuchung zu bilanzieren:

- **Architekturvergleich:** Beide Architekturen können den solaren Verlauf präzise abbilden. Teilweise zeigt das LSTM aufgrund seiner komplexen Gating-Struktur eine höhere Robustheit bei der Speicherung langreichweitiger meteorologischer Abhängigkeiten, was sich in einer stabileren Konvergenz im Training widerspiegelt.
- **Einfluss von Datenvolumen und Auflösung:** Durch die zeitliche Granularität und die Datenmenge wird die Schärfe des Unsicherheitsbandes maßgeblich bestimmt. Die grobe 60-Minuten-Auflösung führt zu signifikant breiteren Bändern, während die Erweiterung der Datenbasis eine präzise Verengung des Sicherheitskorridors bewirkt. Das Modell passt die statistische Unsicherheit dynamisch an den Informationsgehalt der Daten an.
- **Erklärbarkeit:** Mittels Permutation Feature Importance wurde die mathematische Black-Box-Struktur der tiefen neuronalen Netze erfolgreich aufgebrochen. Der Nachweis, dass neben den solaren Basis-Features auch multivariate Wechselwirkungen (z. B. die physikalisch begründete, schwache negative Korrelation zur Windkraft) den Verlust steuern, schafft die benötigte Transparenz für den realen Handelseinsatz.

Mit dem entwickelten System können wetterbedingte Planungsunsicherheiten im Stromnetz mathematisch exakt quantifiziert werden. Die Kombination aus Docker-Containerisierung, Streamlit-Oberfläche und interpretierbaren Machine-Learning-Strukturen bietet die valide Grundlage, um den Bedarf an kostenintensiver Ausgleichsenergie effektiv zu minimieren.

6 Ausblick

Um das Projekt zukünftig fortzuführen, können folgende Punkte benannt werden:

1. Einbezug realer meteorologischer Wetterdaten

Die im Rahmen dieses Modells generierten Prognosen ersetzen keine realen Wetterdaten. PV ist ein physikalischer Prozess, der stark vom Wetter abhängt. In einem nächsten Schritt wäre es daher sinnvoll echte Wetterdaten zwangsläufig miteinzubeziehen und die Prognosen basierend auf Wetterdaten zu verfeinern.

2. Von der Prognose hin zu automatisierten Handlungsempfehlungen durch Sprachmodelle

In das Modell könnte künftig ein Large-Language Modell (LLM) integriert werden, dass bereits eine Vorabinterpretation der Daten vornimmt und dem Nutzer bereits erste Handlungsvorschläge präsentiert. Es muss allerdings beachtet werden, dass ein LLM den Menschen nicht komplett ersetzen darf. Eine vorsichtige Eigeninterpretation der generierten Daten sollte weiterhin stattfinden.

7 Literatur

- [1] T. Hong, P. Pinson, S. Fan, H. Zareipour, A. Troccoli, R. J. Hyndman, *Int. J. Forecast.* **2016**, 32, 896–913.
- [2] D. Salinas, V. Flunkert, J. Gasthaus, T. Januschowski, *Int. J. Forecast.* **2020**, 36, 1181–1191.
- [3] B. Xiong, Y. Chen, D. Chen, J. Fu, D. Zhang, *Appl. Energy* **2025**, 382, 125294.
- [4] B. Lim, S. Ö. Arik, N. Loeff, T. Pfister, *Int. J. Forecast.* **2021**, 37, 1748–1764.
- [5] Bundesnetzagentur, “SMARD,” can be found under <https://www.smard.de/home/downloadcenter/download-marktdaten/>, **2026**.